C++

PENG

October 12, 2026

Contents

1 Introduction **1**
1.1 IO . . . 1
1.2 Comments . . . 2
1.3 Flow of Control . . . 2
1.4 Class . . . 2

2 Types **3**
2.1 Compound Types . . . 4
2.2 Qualifier . . . 4
2.3 Data Structure . . . 5

3 string **6**
3.1 Chars in string . . . 6

4 vector **7**

5 Iterators **7**

6 Array **8**
6.1 Pointers and Arrays . . . 9
6.2 Arrays of Arrays . . . 9

1 Introduction

C++ is a high-level, statically typed programming language [1].

1.1 IO

Definition 1.1 (IO objects)**.**

- `iostream` *: std lib*
- `std::cin` *:* `istream` *type*
- `std::cout` *:* `ostream` *type*

```
std::cout << "str";         // return object: std::cout
std::cout << "str" << "\n"; // (std::cout << "str") << "\n";

int v1 = 0, v2 = 0;
std::cin >> v1;            // return object: std::cin
std::cin >> v1 >> v2;      // (std::cin >> v1) >> v2;
```

Definition 1.2 (namespace)**.**

```
namespace::name // :: scope operator

// headers file should not include using declarations
using namespace::name;
```

1.2 Comments

Definition 1.3 (comment)**.**

```
// single line, best way
/* paired, can not nest */

std::cout << /* "*/" /* "/*" */;
// remove everything between */ and closest /* before
// std::cout << " /* ";
```

1.3 Flow of Control

Definition 1.4 (while)**.**

```
while(cond)
{
    statements;
}

while(std::cin >> val)
// std::cin >> val return left operand: std::cin
// istream become invalid:
// 1. reading a non-integer val: 1 2 3.1
// 2. end-of-file: Ctrl-Z then RET (windows), or Ctrl-D then RET (linux)
```

Definition 1.5 (for)**.**

```
for (init-statement; cond; expr)
{
    statements;
}

// init-statement:
// local var used only inside for, executed only once
// expr: executed after for body
```

Definition 1.6 (if)**.**

```
if (cond)
{
    statements;
}
else if (cond)
{
    statements;
}
else
{
    statements;
}

if (std::cin >> val)
    std::cout << val << "\n";
// 1 2 3 RET => 1
```

1.4 Class

Definition 1.7 (class)**.** *Define data structures:*

- *name*

- *defined*
- *operation*

```
// #include <std lib>
// #include "header.h"
```

Definition 1.8 (member function)**.**

```
data.func()
```

2 Types

Definition 2.1 (arithmetic types)**.**

```
char        // '\0': null, '\t': horizontal tab, '\n': newline
#include <string>
std::string
int         // unsigned: >= 0
bool        // true for 1 and false for 0
double
```

Definition 2.2 (object)**.** *An object is a region of memory that contain data and has a type.*

Definition 2.3 (declaration and definition)**.**

```
// define: allocate storage and defined only once
// source.cpp
int i = 10;

// declare: specify type and var names
// headers should have guards to be safe to be included multiple times
// header.h
#ifndef HEADER  // true if CLASSNAME not defined
#define HEADER  // preprocessor vars are all uppercase
extern int i;
#endif

// main.cpp
#include "header.h"
i ...
```

Definition 2.4 (scope)**.** *Delimited by* `{}`*.*

```
// vars defined outside any function body are initialized to 0
int val; // 0
int main ()
{
    int val = 1; // block scope
    // ::val == 0 global scope
    ...
}

// var: lowercase
// class: first letter uppercase
```

Definition 2.5 (bit)**.**

```
bit      combination         total         number
1 bit:   0, 1;               2     = 2^1;  0, 1
2 bits:  00, 01, 10, 11;     2*2   = 2^2;  0, 1, 2, 3
3 bits:  000, 001, ..., 111; 2*2*2 = 2^3;  0, 1, 2, 3, ..., 7
32 bits:                           = 2^32; 0, 1, 2, ..., 2^32 - 1
```

Definition 2.6 (type conversion).

```
unsigned i = -1; // 2^32 - 1 for 32-bit int

// when mix int and unsigned, int is automatically converted to unsigned
unsigned a = 1;
int b = -1;
a + b // 1 + (2^32 - 1) = 2^32 = 0
a * b // 1 * (2^32 - 1)
a < b // 1 < (2^32 - 1), always true
```

2.1 Compound Types

Definition 2.7 (reference). *A reference defines an alias for an object.*

```
int i = 0;
int &ref = i;
// ref must be bound only to an int object, not a literal
// error: int &ref = 1;
```

Definition 2.8 (pointer). *A pointer is an object that holds the address of another object.*

```
int i = 0, j = 1;
int *p, *p2;
p = &i;      // p holds the address of i
*p = 2;      // return the object i to which p points
p = nullptr; // or p = 0, null pointer, not bound to any object
p = p2;      // copy address
p2 = &j;     // p value not change

// pointer to pointer to ...
int **pp;    // * indicate pointer level
pp = &p;

// ref to pointer
int *&ref = p; // & => ref is reference, * => ref refers to pointer type
```

2.2 Qualifier

Definition 2.9 (const). *The compiler will replace the const var with its value.*

```
// const must be initialized
const int i = 1;

// const var local to a file by default
// must specify extern to be used in other files
// source.cpp
extern const i = 10;
// header.h
extern const i;
// main.cpp
#include "header.h"
i ...

// ref to const
// cannot change the object to which it bounds
const int i = 1;
const int &r = i;
const int &r2 = 2; // compiler: const int temp = 2; const int &r2 = temp;
```

```
// pointer to const
// cannot change the object to which it points
const int i = 1;
const int *p = &i;

// const pointer
// the address it holds cannot be changed
int i = 1;
int *const p = &i; // p always points to i
```

Definition 2.10 (constant expr). *A constant expression is an expr whose value cannot change at compile time.*

```
constexpr int i = size();   // ok only if size is a constexpr function
constexpr int *p = nullptr; // const pointer, constexpr applies to p
```

Definition 2.11 (type alias). *A synonym for a type.*

```
typedef int base; // or: using base = int;
typedef base *p;  // type: int*, pointer to int
p p1 = &i;

// const p: const modify the given type,
// constant pointer to int: int *const
const p p2 = 0; // int *const p2 = 0
const p *p3;    // int *const *p3
```

Definition 2.12 (const level).

```
// top-level const: an object itself is const, i.e. const pointer
const int i = 1;
int *const p1 = &i;

// low-level const: point or refer to const object
const int *p2 = &i;
const int &r = i;

// when copying, top-level const is ignored,
// low-level const is never ignored
const int *const p3 = &i;
p2 = p3;
```

Definition 2.13 (auto). *Deduce the type from the initializer at compile time.*

```
auto i = 1, *p = &i;   // must have initializer
auto i = 0, pi = 3.14; // error, types must be consistent

// auto may different from initializer's type
// auto ignores top-level const
```

Definition 2.14 (decltype). *Return the operand type without evaluating the expr.*

```
decltype(expr) var
```

2.3 Data Structure

Definition 2.15 (structure). *A data structure is a way to group together replated data elements and for using those data.*

```
// class is ordinarily defined outside functions, usually in className.h
struct className
{
    statements;
}; // must end with ;

className x, *y;
```

3 string

Definition 3.1 (string)**.** *A* `string` *is a variable-length sequence of characters.*

```
#include <string>

std::string str = "A";
// copy of the string literal "A": ['A', '\0'],
// not include null char '\0' at the end

// read string
std::cin >> str1 >> str2;

// read line
std::getline(std::cin, str)

// empty
str.empty() // true for empty

// size
str.size() // unsigned type: string::size_type
// str.size() < -1: true =>
// str.size() < 2^32 - 1 for 32-bit int
// do not use int in size() expr

// compare
std::string str = "Hello";
str < "Hiya" // true, compare ASCII of first diff letter: i > e
str < "Hello World"

// cannot compare string literals directly,
// i.e. "a" < "b", which will compare their mem addresses
// at least one should be std::string type

// add
str + " World\n" // "Hello World\n"
```

type conversion: Definition 2.6

3.1 Chars in string

Definition 3.2 (for range)**.**

```
for (char c : str)
{
    statements;
}

// ispunct(char), true if char is punctuation

// define loop var as ref type to change char in string
for (char &c : str)
    c = toupper(c); // uppercase

// index: str[i]
// 0 <= i <= str.size() - 1
// i is unsigned string::size_type
// decltype(str.size()) i = 0;
// string::size_type i = 0;
// str must not be empty:
```

```
// !str.empty()
for(decltype(str.size()) i = 0; i != str.size(); i = i + 1)
    ...

// isspace(char), true if char is space
```

4 vector

Definition 4.1 (vector)**.** *Collection or container of objects with same type.*

```
#include <vector>
std::vector<type> v;

// vector is a class template
// template: class or function templates
// template is instantiation that the compiler use to create classes or functions

// initialization: list initialize or elemcount
std::vector<int> v(n);               // n eles = 0
std::vector<int> v(n, val);          // n eles = val
std::vector<int> v{val1, ..., valn};
// list initialize, ele1 = val1, ... , elen = valn if possible

// if list initialization is not possible,
// the compiler looks other ways to initialize the object from the given value.
std::vector<string> v{n};
// n is not string, list initialize false, use elemcount instead: n eles = empty
std::vector<string> v{n, "str"}
// list initialize false, use elemcount instead: n eles = "str"

// add elemto vec
// v.push_back(ele)

// elemin vec
for(type &r : v)
    ... // assign new value
for(type i : v)
    ...

v.empty(), true if vec is empty
v.size(), return type: vector<type>::size_type

// compare

// index
// !v.empty()
// 0, ..., v.size() - 1
```

5 Iterators

Definition 5.1 (iterator)**.** *Access the elemin string or vector.*

```
auto b = v.begin(); // denotes first ele
auto e = v.end();   // denotes one after the last ele

// for empty vec
// b == c
```

```
// access ele
// *b = toupper(*b);

// move iterator
++b // advance the iterator by one position
--e // previous ele

// compare iterators with == or !=
for(auto it = v.begin(); it != v.end(); ++it)
    ...

// iterator type
// std::vector<int>::iterator
// std::string::iterator
// std::vector<int>::const_iterator
// cannot write elemin vec
// std::string::const_iterator
// cannot write char in string
auto b = v.cbegin(); // const_iterator
auto e = v.cend();

// deref and member access
// (*it).mem_func() <=> it->mem_func()

// iterator arithmetic for vector and string
// it +/- n
// it1 <= or >= it2: ele1 appear before or after ele2
auto mid = b + (e - b) / 2;
```

6 Array

Definition 6.1 (array)**.** *Array is a compound type and fixed-size container of unnamed objects of a single type that we access by position.*

```
type array_name[dim]; // dim = const expr > 0

// default-initialized array defined inside a function has undefined values

// list initialize
int a[3] = {};    // zero-initialization, all elems 0
int a2[3] = {1, 2}; // same as a[] = {1, 2, 0}, rest default to 0
std::string a3[3] = {"hi", "wolrd"}; // std::string a3[] = {"hi", "world", ""};

// char array
char a[] = "c++"; // string literal end with null char
// char a[] = {'c', '+', '+', '\0'};

// no copy assignment
// error: a = a2;

// array of pointers
int a[2] = {};
int *p[2]; // array of 2 pointers to int:
// right array size -> left elem type

// no array of ref
// error: int &r[2] = ...
```

```
// ref and pointer to array
int (*p2)[2] = &a; // p2 is a pointer to array of 2 int:
// () pointer -> right array size -> left elem type
int (&r)[2] = a;  // r refers to array of 2 int
// () ref -> right array size -> left elem type
int *(&r2)[2] = p;
// () ref -> right array size -> left elem type: pointer to int

// access elem in array
// array subscript type: size_t, unsigned
constexpr size_t array_size = 10;
int a[array_size] = {};
```

6.1 Pointers and Arrays

```
int a[] = {0, 1, 2};
int *p = a;  // int *p = &a[0];
auto p2 = a; // int *p2 = &a[0];
decltype(a) a2 = {0, 1, 2}; // int a[3] = {0, 1, 2};

// iterator
++p // &a[1]
int *e = &a[3]; // off-the-end pointer
(for int *b = a; b != e; ++b)
    ...

int *b = std::begin(a); // pointer to 1st elem
int *e = std::end(a);   // pointer to one past last elem

// arithmetic
int *e = a + 3;                       // points to one past last elem
auto n = std::end(a) - std::begin(a); // type: ptrdiff_t
int last = *(b + 2)                   // a[2]

// subscript
int *p = &a[2];
// p[1]:  *(p + 1): one past last elem
// p[-2]: *(p - 2): a[0]

// vectors and arrays
```

6.2 Arrays of Arrays

Definition 6.2 (multidimensional array)**.** *Arrays of arrays.*

```
constexpr size_t r=..., c=...;
int a[r][c] = {0};

// for range
for (auto &row : a)
    for(auto col : a)
        ...
// all control vars except innermost array must be ref

// pointers
int *(p)[c] = a;
auto *p = a;     // int (*p)[c] = &a[0];
auto q = *p;     // int *q = &a[0][0];
```

```
// type alias
using int_array = int[r];
typedef int int_array[r];
```

References

[1] S.B. Lippman, J. Lajoie, and B.E. Moo. *C++ Primer*. Pearson Education, 2012. ISBN: 9780133053036.